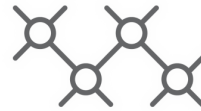




TECHNISCHE
UNIVERSITÄT
WIEN



Institut für
Computertechnik
Institute of
Computer Technology

A BACHELOR THESIS ON

Platform Independent Rendering Algorithm for Microcontrollers

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS

FOR THE DEGREE OF

Bachelor of Science

in

Electrical Engineering and Information Technology (033 235)

by

Dominik Pusztai

12009186

Supervisor(s):

Univ.Prof. Dipl.-Ing. Dr.techn. Axel Jantsch

Univ.Prof. Dipl.-Ing. Dipl.-Ing. Dr.techn. Michael Wimmer

Vienna, Austria

October 2024

Abstract

The process of generating two-dimensional images from a three-dimensional description, commonly called 3D-rendering, has found applications across industries and entertainment. While its demand has brought powerful and moderately cheap dedicated hardware specifically designed for this task, rendering on extremely resource-constrained embedded systems and MicroController devices is not easily accessible without major development effort. Running any computationally expensive task on these platforms demands focused design and optimization to best utilize their limited capabilities, which lies in stark contrast to the adaptability a universal rendering solution must possess to be usable for a wide range of applications. This thesis develops such a solution in the form of an open-source C++ library titled *MicroRenderer*, aiming to enable simple and quickly implementable 3D rendering for projects running on low-performance systems, without requiring extensive software development by the user. To deliver the necessary flexibility for generating diverse visual appearances of 3D objects, *MicroRenderer* builds on a set of user-implemented classes to provide shading-code, which is called without incurring any additional overhead by a sophisticated template-based renderer design. This approach further provides elegant access for interpolating values over 3D-object surfaces, a key requirement to allow for arbitrary shading computations. In combination with a set of featured math classes, which allow a user to quickly optimize major operations in rendering for a specific device and instruction set, *MicroRenderer* provides simple, flexible and simultaneously overhead-free access to 3D rendering on embedded systems, analogous to rendering with dedicated hardware.

Kurzfassung

Die Generierung eines zwei-dimensionalen Bildes aus einer drei-dimensionalen Beschreibung, bekannt unter dem Begriff 3D-Rendering, hat vielfache Anwendungen in der Industrie bis zur Unterhaltungsbranche gefunden. Während hohe Nachfrage leistungsstarke und verhältnismäßig günstige Hardware speziell für diese Aufgabe hervorgebracht hat, ist Rendering auf extrem ressourcenbegrenzten Embedded Systems und Mikrocontrollern nicht leicht zugänglich, ohne dafür wesentlichen Entwicklungsaufwand zu betreiben. Jegliche rechenintensiven Prozesse auf derartigen Plattformen verlangen fokussiertes Design und Optimierung, um deren begrenzte Leistungsfähigkeit bestmöglich auszunutzen, was in starkem Kontrast zur notwendigen Adaptierbarkeit einer universell einsetzbaren Lösung für 3D-Rendering für diverse Anwendungen steht. Im Zuge dieser Arbeit wird ein 3D-Renderer in Form einer Open-Source C++ Bibliothek unter dem Namen *MicroRenderer* entwickelt, mit dem Ziel, einfaches und schnell implementierbares Rendering für Projekte auf leistungseingeschränkten Systemen zu ermöglichen, ohne extensive Software-Entwicklungsarbeit seitens des Nutzers zu erfordern. Um die notwendige Flexibilität zur Generierung unterschiedlichster visueller Darstellungen von 3D-Objekten bereitzustellen, baut *MicroRenderer* auf eine Reihe von Nutzer-implementierten Klassen, um Shading-Code bereitzustellen, welcher durch eine spezielle, Template-basierte Rendererarchitektur ohne zusätzlichen Overhead ausgeführt wird. Des Weiteren stellt dieser Ansatz eine elegante Möglichkeit zur Interpolierung von Werten über 3D-Objekt Oberflächen, eine Schlüssel-Voraussetzung, um beliebige Shading-Berechnungen zu erlauben. In Kombination mit einer Reihe von Mathematik-Klassen, die dem Nutzer erlauben schnell weitgehende Optimierung in den Rendering-Prozessen für spezifische Plattformen und Befehlssatzarchitekturen durchzuführen, bietet *MicroRenderer* einfachen, flexiblen und gleichzeitig Overhead-freien Zugang zu 3D-Rendering in Embedded Systems, analog zu Rendering mit dedizierter Hardware.

Erklärung

Hiermit erkläre ich, dass die vorliegende Arbeit ohne unzulässige Hilfe Dritter und ohne Benutzung anderer als der angegebenen Hilfsmittel angefertigt wurde. Die aus anderen Quellen oder indirekt übernommenen Daten und Konzepte sind unter Angabe der Quelle gekennzeichnet.

Die Arbeit wurde bisher weder im In- noch im Ausland in gleicher oder in ähnlicher Form in anderen Prüfungsverfahren vorgelegt.

Copyright Statement

I, Dominik Pusztai, hereby declare that this thesis is my own original work and, to the best of my knowledge and belief, it does not:

- Breach copyright or other intellectual property rights of a third party.
- Contain material previously published or written by a third party, except where this is appropriately cited through full and accurate referencing.
- Contain material which to a substantial extent has been accepted for the qualification of any other degree or diploma of a university or other institution of higher learning.
- Contain substantial portions of third party copyright material, including but not limited to charts, diagrams, graphs, photographs or maps, or in instances where it does, I have obtained permission to use such material and allow it to be made accessible worldwide via the Internet.

Signature: _____

Vienna, Austria, October 2024

Dominik Pusztai

Contents

Abstract	iii
Kurzfassung	iv
1 Introduction	1
1.1 Motivation	1
1.2 Project goals	2
2 Theory	3
2.1 Overview	3
2.2 Object Representation	4
2.3 Object-order rendering in-depth	4
2.4 Programmability of GPUs	8
3 Implementation	9
3.1 MCU-specific considerations	9
3.2 Rendering overview	11
3.3 Shader framework	12
3.4 Attribute interpolation	14
3.5 Input/Output data representation	16
3.6 Pipeline in-depth	17
3.7 Library Usage	24
4 Evaluation	27
4.1 Test platforms	27
4.2 Test procedure	28
4.3 Test Shader 1: SimpleContours	28
4.4 Test Shader 2: UnlitTextured	30

4.5 Analysis	32
5 Conclusion and Outlook	35
Bibliography	37
A Configuration Structures	39
A.1 RendererConfiguration	39
A.2 ShaderConfiguration	39
A.3 TextureConfiguration	40

List of Tables

4.1	Measured frame times of the SimpleContours Shader test scene.	29
4.2	Measured frame times of the UnlitTextured Shader test scene with texture and shader output type <i>TYPE_NORMALIZED</i>	31
4.3	Measured frame times of the UnlitTextured Shader test scene with texture and shader output type <i>TYPE_INTEGER</i>	31

List of Figures

2.1	Overview of the basic stages of the rendering pipeline. Simplified from <i>Real-Time Rendering</i> [1], pp. 12-25	5
2.2	The view-volume and plane designations of a perspective projection.	6
2.3	Visualization of perspective projection.	6
3.1	Visualization of non-trivial clip cases. Resulting triangles are colored in green.	19
3.2	The coordinate system in screen-space assumed by <i>MicroRenderer</i>	20
4.1	Renders of the SimpleContours Shader test scene, captured on PC. The left image was rendered at resolution 1000x1000. The right image was rendered at resolution 120x120.	30
4.2	The textures used in the UnlitTextured Shader test scene, visualized in 24-bit RGB.	31
4.3	Renders of the UnlitTextured Shader test scene, captured on PC. The left image was rendered at resolution 1000x1000. The right image was rendered at resolution 120x120. Slight distortion in the applied textures are due to linear interpolation of UV coordinates in perspective screen-space.	32

Acronyms

CPU Central Processing Unit. 1, 8, 10, 22, 28

DMA Direct Memory Access. 12

DSP Digital Signal Processing. 10

fps Frames Per Second. 2, 28, 32

FPU Floating Point Unit. 9, 27

GPU Graphics Processing Unit. vii, 1, 2, 4, 7, 8, 12, 35

IDE Integrated Development Environment. 14

MAC Multiply–Accumulate. 10

MCU Microcontroller Unit. vii, 1, 2, 9, 10, 11, 12, 16, 27, 28, 32, 35

RAM Random Access Memory. 9, 10, 12, 27, 32

ROM Read Only Memory. 16, 17, 27

SIMD Single-Instruction, Multiple-Data. 10

SoC System-on-a-Chip. 1

Chapter 1

Introduction

1.1 Motivation

Three-dimensional visualizations are no longer restricted to expensive computing systems nowadays, as even low-cost devices like smartphones are equipped with custom 3D rendering hardware integrated together with the Central Processing Unit (CPU) in the form of System-on-a-Chip (SoC) packages. On even more resource-constrained systems lacking graphics-dedicated hardware, like the majority of Microcontroller Units (MCUs), even simple 3D graphics are still difficult to achieve and therefore often avoided. However, steady improvements in the performance of MCUs, as well as their omnipresent use in industrial as well as household devices, make the computations of very simple 3D graphics on them attractive.

The almost ubiquitous use of dedicated hardware for 3D rendering has led to the development of multiple standardized graphics APIs¹, facilitating communication between the host CPU and the so-called Graphics Processing Unit (GPU). These APIs, usually unanimously supported by all major GPU manufacturers and also often operating-system (if any) independent, allow for simple and portable access to programmable 3D rendering. Similar access to purely software-based rendering, on the other hand, is rare, as existing frameworks are most often closed-source and targeted for more capable devices. Solutions suitable for the performance of MCUs are usually highly specialized for a specific application, not standalone and not easily adaptable to another use-case. Subsequently, any project wishing to incorporate even extremely simple 3D graphics, like a coffee machine showcasing a floating company logo, would need to implement a full rendering pipeline from scratch. This thesis seeks to provide a simpler and flexible solution.

¹Examples include DirectX, OpenGL, Metal, etc.

1.2 Project goals

A C++ header-only library shall be developed to enable quick and simple 3D rendering on 32-bit MCU-based devices.

Usage of this library must not require any specific 32-bit MCU, nor any specific manufacturer and shall also not assume any additional external hardware (including display controllers). The library shall be self-contained with no external dependencies, to facilitate easy integration into other projects and to avoid unnecessary code bloat.

In analogy to programmable GPUs, shading² computations shall not be fixed into the renderer, but be easily implementable by the user to specifically fit their application. The design of an efficient, easy to work with framework allowing for this flexibility without introducing any computational overhead is a core goal of the project, and a distinct difference to existing solutions.

While these requirements prevent utilizing any device-specific instructions or peripherals for optimization, a clear library structure and good documentation should assist the user in modifying the code for their specific environment on their own, if necessary.

The broad set of potential platforms makes it difficult to define a concrete performance target, but the aim is to make image generation fast enough to compute real-time video of simple scenes³ on low-end processor cores, like the Arm Cortex-M0 [2].

The library has been given the name *MicroRenderer* and will be referred to by this name for the rest of the thesis. *MicroRenderer* is published under the MIT license and available at <https://github.com/ronypepper/MicroRenderer>.

²*Shading* in the context of computer graphics refers to operations performed during rendering that influence the scene's depiction in the final image. In modern GPUs, these can be supplied by the user as code scripts, called *Shaders*, and stand in contrast to other fixed functions in the rendering process. See Section 2.3.

³Real-time shall be defined as at least 20-24 Frames Per Second (fps). A simple scene is assumed to contain no more than a few dozen triangles to render.

Chapter 2

Theory

A basic overview of common approaches to 3D rendering is presented. After discussing the representation of objects, the focus is laid on one particular approach most suited for implementation in *MicroRenderer*, and the individual steps within it are covered in more detail. Most of the information presented in this chapter is based on *Real-Time Rendering* [1] and *Fundamentals of Computer Graphics* [3].

2.1 Overview

The process of rendering performs the task of converting a description of a three-dimensional scene into a two-dimensional image, as seen from a specified point of view. The description contains at least information defining the objects to be rendered (their surface, also called model), as well as their spatial position, orientation and scale. A variety of additional information may be included, like lights illuminating the scene or textures applied to models, depending on the computations that are performed to determine the visual appearance of objects.

A core problem in rendering is to determine what pixels are covered by which objects. Two general approaches can be taken to solve it: With *image-order rendering*, individual pixels are considered in turn, determining for every object of the scene if it intersects the respective pixel. *Object-order rendering* on the other hand considers objects in turn, and determines for every object individually the set of pixels it covers. These two approaches both possess advantages and disadvantages, depending on the desired scene to be rendered, but *image-order rendering* is generally associated with greater (easier) potential to render advanced effects, while also being computationally more demanding. For the latter reason, discussed further in Section 3.2, *MicroRenderer* is implemented using *Object-order rendering*.

2.2 Object Representation

The surface of objects can be defined in variety of ways, for example as an implicit or a parametric equation, or by a set of geometric primitives, like polygons. In practice most GPUs and many software solutions only support rendering points, lines and triangles as geometric primitives, with complex 3D models constructed using a large set of triangles¹. Reason for this is that triangles have some beneficial mathematical properties, like barycentric coordinates and guaranteed planarity, allowing them to be efficiently processed while rendering. Furthermore it simplifies the design of both GPUs and software renderers.

Subsequently *MicroRenderer* is also limited to rendering triangles only.

A set of triangles forming an object is called a (*triangle-*)*mesh*. A single triangle is represented by the position of its three corners, called *vertices*, which may be shared among the members of a mesh. The data representing a single mesh is usually structured as at least two lists: a *vertex-list*, storing vertex positions in an object-local coordinate frame; and an *index-list*, which stores indices into the vertex-list. Three consecutive indices define a triangle, thus reducing storage requirement by reusing vertex information. Any number of further lists may be included for additional per-vertex data, for example normals or colors, as required for computing an object's visual appearance.

2.3 Object-order rendering in-depth

Object-order rendering occurs in a series of stages, which are collectively referred to as the *rendering pipeline*. The exact composition of the pipeline depends on the (hardware or software) architecture of the system in question, but it can be broadly described in four main stages as shown in Figure 2.1.

2.3.1 Geometry Processing

All processes performed here operate on geometric primitives, i.e., vertices and triangles.

Vertex Shading

Receiving the input data describing the scene from the host system, the rendering pipeline starts with performing a set of computations once for every vertex of the entire scene, referred to as *Vertex Shading*. These depend on the desired final image, but generally include a transformation of the vertex position, which takes it from their initial coordinate-frame (local to their respective mesh) called the model-space, to a normalized space called clip-space. Homogeneous coordinates are utilized to perform the necessary affine transformations with a simple matrix-vector multiplication.

¹Series of triangles, e.g., strips or fans, are also commonly supported.

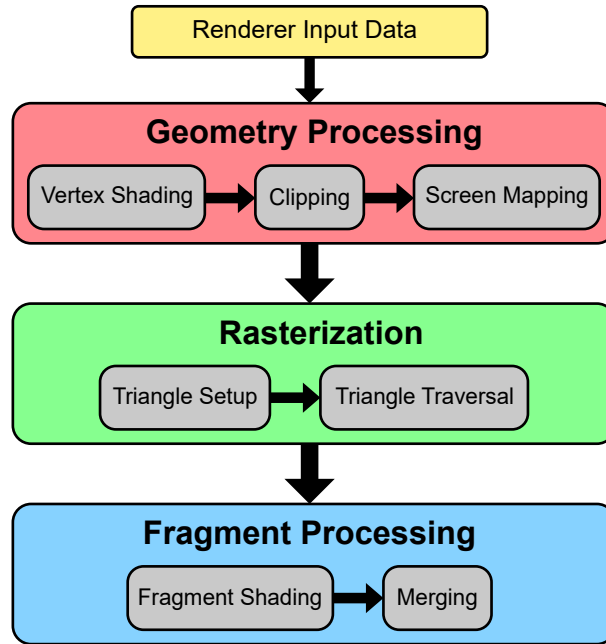


Figure 2.1: Overview of the basic stages of the rendering pipeline. Simplified from *Real-Time Rendering* [1], pp. 12-25

The transformations applied in order are:

1. Model-space to world-space: applies the associated object's world position, rotation, scale.
2. World-space to view-space: applies the view-point's position and the view direction.
3. View-space to clip-space: squashes the desired *view-volume*, which contains the space visible in the image, into a normalized cube².

The last transform corresponds to a *projection*, which can be either *orthographic* or *perspective*. The starting view-volume of the former is a rectangular cuboid, so the transform is a simple translation and scaling operation. Perspective projection however uses a pyramid that has been cut off at the tip as its view-volume, defined by its six planes, see Figure 2.2, which corresponds to a transformation that cannot be performed as a linear matrix-vector multiplication. In perspective, objects farther away appear smaller, a relation which can be modeled as a division of a point's coordinates on the image plane by its distance from the view-point, see Figure 2.3:

$$y_{projection} = \frac{n}{z} y_{view}$$

This division needs to be applied separately, however the homogeneous component can be exploited to preserve the distance from the view-point, commonly referred to as the *depth*, during the view-space

²Different clip-spaces are possible, e.g., a cube spanning $[-1;-1;-1]$ to $[1;1;1]$ or $[0;0;0]$ to $[1;1;1]$. Transformation into a unit-cube simplifies the following step of clipping.

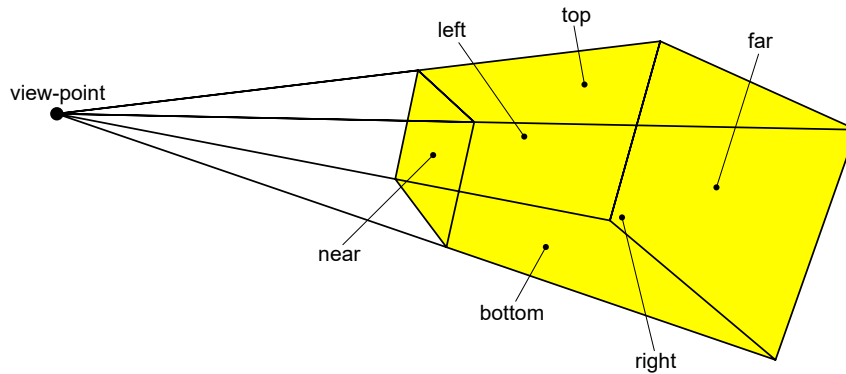


Figure 2.2: The view-volume and plane designations of a perspective projection.

to clip-space matrix-transform.

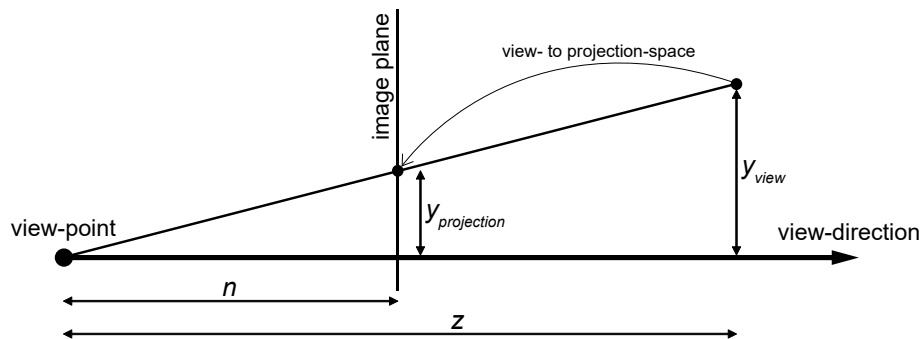


Figure 2.3: Visualization of perspective projection.

Clipping

To avoid processing unseen geometric primitives, they are now tested against the bounds of the transformed view-volume. If primitives are fully outside it, they are discarded. If they intersect the bounds, they are *clipped* (cut) against them, i.e., new triangle(s)³ are created for the part inside the view-volume. It is necessary to perform clipping before perspective division, as invisible points lying behind the view-point are mapped beyond the view-volume in view-direction, subsequently leading to incorrect results. This effect can be seen in *Fundamentals of Computer Graphics* [3], p. 190.

Screen Mapping

Next the vertex positions are simply shifted and stretched to fit pixel coordinates, so that the clip-space now covers the screen's resolution. The depth component is still needed later and left unchanged.

³Cutting a triangle by a straight line may produce a quad, which needs to be split into two triangles for further processing.

2.3.2 Rasterization

This stage performs the task associated with object-order rendering: determining the set of covered pixels for each triangle, or in other words, converting the geometric representation of objects to discrete pixels on screen. It may be solved by constructing and using edge-equations to test a number of pixel candidates (e.g., the pixels inside the triangle's bounding box), each on their own. One method would be for example to compute the pixel's barycentric coordinates and check $\alpha, \beta, \gamma \geq 0$.

A different approach is to pick a start point on the triangle and then "walk" across it. This methodology is applied in a whole family of algorithms called *scanline algorithms*, which rasterize polygons in *scanlines*, i.e., horizontal or vertical pixel rows, while trying to exploit the coherence between adjacent lines. A summary of selected solutions developed over the years can be found in a paper from *ICCSA 2007 Proceedings, Part II* [4]. Historically scanline algorithms have been preferred initially, but the parallelizable nature of edge-equation based approaches have made them the dominant implementation in GPUs.

A closely related problem is that of *hidden-surface removal*, which aims to ensure that for every pixel, the closest object covering it is drawn. This may already be implemented along rasterization, as done in many scanline algorithms by performing a depth-sort/intersection of triangle spans per scanline, or may be decided on a pixel-by-pixel basis in the *Fragment Processing* stage (Section 2.3.3).

Independently of the chosen approach, the *Rasterization* stage can be divided into two sections as follows.

Triangle Setup

Here any pre-processable information required by the chosen rasterization procedure is computed and stored. Usually at least barycentric coordinates for a start pixel are calculated, which will be used to interpolate *attributes*⁴ across the triangle.

Triangle Traversal

The chosen rasterization procedure is performed here to determine the pixels covered by the triangle. Any number of attributes required in per-pixel shading computations are interpolated to those pixels, e.g., position, normal, color, etc.

2.3.3 Fragment Processing

All processes performed here operate on individual pixels of the image.

⁴Any numeric value belonging to a vertex is called an *attribute* of the vertex. These may be supplied with the input object data, or computed during Vertex Shading.

Fragment Shading

Any number of computations may be performed here for every pixel covered by a triangle, depending on the desired appearance of objects in the final image.

Merging

This last stage solves hidden-surface removal, if not already done during rasterization, using a technique called *z- or depth-buffering*. It requires additional memory for a *depth-buffer*, in which the depth of every drawn pixel in the image is stored, i.e., its distance from the image-plane. Pixels are now only drawn into the image if their depth is smaller than the respective entry in the depth-buffer, updating the stored depth if so. The depth-buffer needs to be appropriately filled at the start of rendering. This check, called the *depth-test*, may be performed before Fragment Shading to avoid unnecessary calculations, a practice referred to as *Early-Z*.

A few other operations may in practice be performed here as well, for example *blending* a new pixel together with the one previously in the image. These are omitted in *MicroRenderer* and not further discussed.

2.4 Programmability of GPUs

The Vertex and Fragment Shading sub-stages described above can be controlled in modern GPUs, by providing scripts of code to the hardware, called *Vertex Shader* and *Fragment Shader* respectively⁵. The Vertex Shader receives per-vertex data as input, previously supplied by the host CPU, and can output data to be interpolated across the triangle, which is then received as per-pixel data by the Fragment Shader. The Vertex Shader is also responsible for outputting a transformed vertex's position in homogeneous clip-space, which is used for clipping and rasterizing triangles by fixed-function hardware in the GPU. The Fragment Shader outputs the color to be drawn in the image. Both Shaders can also have access to *uniform data*: constant data uniform during the render process.

This programmability makes it possible to implement a wide range of effects, different in visual appearance and computational complexity, and is replicated in *MicroRenderer* with a special set of classes called the *Shader Framework*.

⁵A modern GPU can receive more Shaders. These additional features are omitted in *MicroRenderer* and not further discussed.

Chapter 3

Implementation

Special considerations applying to the targeted MCU platforms are discussed first, before giving an overview of *MicroRenderer*'s rendering pipeline. Specifics of the implementation are explained next, including a *Shader Framework* suited for modification by the user, efficient interpolation of values over triangles, and scene data representation. The pipeline is then presented in detail, including a custom scanline rendering algorithm developed for *MicroRenderer*. Finally, short remarks about library usage are given.

3.1 MCU-specific considerations

Modern MCUs have become powerful computing devices, with rich instruction-sets, fast Floating Point Units (FPUs) and clock speeds commonly in the hundreds of Megahertz, while coming at a very low price point. Still, a few specific hardware limitations need to be bared in mind:

- While 1-cycle hardware multipliers are widely available in MCUs, even in "low-end" cores like the Arm Cortex-M0 [2, Section 3.3], hardware dividers often only provide multiple-cycle operations, for example a single division on an Arm Cortex-M4 can take up to 12 cycles.¹ [5, Section 3.3.1]. This also applies to hardware FPUs [5, Section 7.2.3]. Considering the limited performance budget and less possibility for the compiler to hide this latency due to simpler processor architectures, use of division operations should be minimized.
- Available Random Access Memory (RAM) on MCUs is especially constrained compared to clock frequency. An RP2040 for example, a low-cost 32-bit MCU by the Raspberry Pi Foundation, boasts dual Arm Cortex-M0+ cores clocked at 133MHz², while providing 264kB of on-chip RAM

¹Arm's M-architecture line of processors are used as examples due to their explicit design for use in MCUs and embedded systems, and their popularity.

²Easily overclockable to 250MHz, as used for evaluation in Chapter 4.

[6, Chapter 1]. For comparison, the possibly first commercial true-3D polygonal video game, *Descent*, developed by Parallax Software Corp. in 1995, required at least a 386-33 CPU combined with 4Mb of RAM [7]. That is an about four times slower clock-rate than the RP2040 (which additionally has double the cores), but 16 times its available memory! As computer graphics is generally a memory-hungry application, saving RAM wherever possible is of utmost importance.

- Despite the meager amounts of available RAM (comparatively, of course), it can still be large enough on more powerful MCUs for them to incorporate cached memory-architectures, like the STM32H747 by STMicroelectronics [8]. Structuring object input-data, among others, in a cache-friendly way is therefore desirable.
- Many small operations like vector and matrix math, transformations, interpolation, etc. are very frequently executed throughout the rendering pipeline, from geometry processing to color computations. These are highly amenable for optimization with special instructions like Single-Instruction, Multiple-Data (SIMD) or Multiply–Accumulate (MAC), which are becoming increasingly available in MCUs, especially since they are often in limited form part of Digital Signal Processing (DSP) extensions [9]. Some MCUs also offer specialized peripherals to accelerate a range of common tasks, like hardware-interpolators on the RP2040 [6, Section 2.3.1.6.].

Since a primary design goal of *MicroRenderer* is portability, special instructions unfortunately must not be utilized in library code. As vector, matrix, transform, and interpolation routines are contained in their own classes however, the user can relatively simply incorporate such optimization specifically for their target devices, potentially greatly increasing render performance.

It is assumed that *MicroRenderer* is run on a 32-bit MCU, since 16-bit architectures would introduce additional numerical problems and are usually even more constrained in their resources.

3.1.1 C++ Templates

C++ templates are heavily used across the library to provide class configurations, the user supplied shading functions to be executed (see Section 3.3) and the underlying numeric data type used in all computations. This seeks to enable the desired flexibility for the libraries usage, while maximizing opportunity for compiler optimization, by enabling inlining of all user-programmed functions. This avoids function call overheads for very frequent routines, e.g., per-pixel operations, but also enables possibility of many other compiler optimizations.

3.2 Rendering overview

The first decision to be made is if *MicroRenderer* should be based on image-order or object-order rendering.

Ray-casting is the usual method associated with image-order, where for every pixel a ray in the appropriate, projection-dependent direction is generated, and every triangle in the scene is tested for intersection with the ray. Real-time rendering a 240x240 resolution image just covered by a single triangle, would amount to $240 * 240 * 20 * 1 = 1.152.000$ required intersection-tests. Even when using an efficient algorithm, like presented by Möller and Trumbore [10], it still requires multiple vector operations and an expensive division to compute intersection. Consequently, image-order techniques still certainly lie beyond the capabilities of today's MCUs. Employing spatial-acceleration structures for culling the triangle test-set early or using iterative techniques like in *Ray-Marching* won't change the situation either.

Therefore, *MicroRenderer* is built on object-order rendering, with a structure similar to what is presented in Section 2.3.

The next question is how to handle rasterization. Taking a scanline-rasterization approach may open up an interesting opportunity for massive memory saving when taking into account the display system. Common displays are driven by feeding them color data at specific intervals. Horizontal lines of pixels are refreshed one after another until the entire image has been received by the display, followed by a time span called *vertical blanking*, before resuming with the first scanline again. Display controllers are used to provide the pixel data to the display with the required timing, reading from a memory block called the *framebuffer*. Many low-cost displays nowadays come with such a controller built-in, complete with its own framebuffer, that receives new image data from a host system, e.g., a MCU, through a parallel or serial communication interface. Usually rendering expects a full framebuffer residing in host system memory to write into, which is copied to the display controller's own framebuffer afterwards. However, if rendering happens ordered in lines, like with scanline-rasterization, one may render and directly send lines to the display controller using just *linebuffers*, big enough to only hold pixel data for one image line. In order to avoid *tearing*, an unpleasant effect where parts of the old and new image are visible on-screen at the same time, the display controller needs to send both *V-Sync* and *H-Sync* signals with this approach³:

Initiate rendering when v-sync occurs. When rendering finishes a new scanline, stop and transmit it as soon as the number of received H-Sync signals since the start of rendering is equal or higher than the number of scanlines rendered. Then resume rendering into the same linebuffer. Performance can

³V-Sync indicates that the entire screen has finished refreshing. H-Sync indicates that a new scanline has finished refreshing.

be improved by transmitting scanlines using Direct Memory Access (DMA) and two linebuffers: while one is being transmitted, the other is rendered into. This approach requires rendering to happen at at least half the display's refresh rate, i.e., one new frame is presented every other refresh cycle. Using additional linebuffers could improve performance if the rendered scene changes in complexity over the image, by allowing rendering simple lines ahead of the display's refresh to buy extra time for more complex lines later on.

MicroRenderer is built on a triangle-specific scanline-rasterizer to enable framebuffer-less rendering. Hidden-surface removal is performed using a depthbuffer, since it is simple to implement and provides access to per-pixel depth values, which may be needed for shading anyways. Big costs in memory are avoided by scanline-rasterization, since the depthbuffer can be supplied as a single linebuffer as well, even when using multiple linebuffers for color. Memory savings with this approach can be dramatic. For example, a 240x240 resolution display in a 16-bit color-mode would require about 345kB for a full framebuffer and a single precision floating point depthbuffer. These on their own would not even fit into the available RAM of a RP2040 MCU. A downside of the rasterization-approach used is the requirement to sort triangles along scanlines.

Usage of linebuffers or a framebuffer is agnostic to the library itself, as rendering simply interrupts after every scanline, letting the user change the targeted memory region as desired. This accounts for systems where no external framebuffer-equipped display controller is in use⁴. The *Renderer* class lets the user even disable scanline-rendering, rasterizing triangles directly into a full framebuffer without sorting and storing them in buffers, as it is naturally supported by the rasterization procedure used (Section 3.6.2).

The core class implementing the rendering pipeline is called **Renderer**. It is templated with:

- A numeric data type used for all internal computations, which must be float or double.
- A *ShaderProgram* class, see Section 3.3.
- An instance of the **RendererConfiguration** structure that configures the implemented pipeline, see Appendix A.1.

This class requires various external buffers for operation, detailed later, which must be set before rendering.

3.3 Shader framework

To achieve a programmability in *MicroRenderer* analogous to GPUs as explained in Section 2.4, the library builds on a set of overridable/specializable template classes to provide user-programmable func-

⁴More powerful MCUs may have this already integrated for example.

tionality, while using a fixed clipper and rasterizer for triangles.

In order to customize image generation, users of the library must override and implement, respectively specialize, the following set of classes, collectively called a **Shader**:

- **BaseVertexShader** is a CRTP⁵ class requiring the user to implement the **shadeVertex** method⁶, which provides ability for custom per-vertex operation.
- **BaseTriangleAssembler** is a CRTP class requiring the user to implement the **interpolateVertices** and **setupTriangle** methods. The former is called by the clipper to interpolate attributes to new vertices, while the latter is called for every triangle passing the clipper to set up attributes for interpolation (and possibly compute any other triangle-specific data).
- **BaseFragmentShader** is a CRTP class requiring the user to implement the **interpolateAttributes** and **computeColor** methods. The former is called by the rasterizer to interpolate attribute data, while the latter provides ability for custom per-pixel operation.
- **BaseShaderInterface** must be specialized with a set of structures that define Shader in- and output, namely:
 - **GlobalData**: all Shader methods receive a constant reference to the same instance of it, which must be supplied to the Renderer class by the user. It poses as global uniform data.
 - **InstanceData**: all Shader methods receive a constant reference to an instance of it, belonging to the object of the vertex/triangle currently being processed. This structure must inherit **BaseInstanceData**, which contains an index into the Renderer's model array, see Section 3.5. They must be supplied as an array by the user, representing the objects to be rendered (their uniform data).
 - **VertexSource**: VertexShader and TriangleAssembler methods receive constant reference(s) to instance(s) of it, supplied as part of model descriptions by the user as per-vertex input data. The interpolateVertices method also receives a mutable reference to a temporary one for writing a new interpolated vertex.
 - **VertexBuffer**: VertexShader and TriangleAssembler methods receive reference(s) to instance(s) of it, providing them with per-vertex mutable data. This structure must inherit **BaseVertexBuffer**, which provides storage for the transformed position. The user must supply the Renderer class with a buffer for storing these, containing at least as many entries as the number of vertices of the model with the largest vertex-count, since they only need to be stored while one model is processed. The interpolateVertices method also receives a

⁵CRTP is short for the Curiously Recurring Template Pattern, a C++ idiom implementing static polymorphism. This avoids expensive run-time virtual table lookups and memory costs of dynamic polymorphism.

⁶All noted methods to be implemented by the user are actually called: `methodName_implementation`.

reference to a temporary one for writing a new interpolated vertex.

- **TriangleBuffer**: FragmentShader methods receive a mutable reference to an instance of TriangleBuffer, providing them with per-triangle object data. The setupTriangle method also receives a mutable reference to one for writing a new triangle. This structure must inherit **BaseTriangleBuffer**, which provides storage for depth interpolation. All instances are stored as part of their RasterizationBuffer structures (Section 3.6.2).

The specialized ShaderInterface acts as a bridge between the other framework classes. It must also be provided in the declaration of the derived VertexShader, TriangleAssembler and FragmentShader⁷

- **BaseShaderProgram** must be specialized with the derived VertexShader, TriangleAssembler and FragmentShader. It acts as a compact representation of the Shader.

All shader classes are templated with:

- A numeric data type used for all internal computations, which must be float or double. Must match the Renderer's numeric data type.
- An instance of the **ShaderConfiguration** structure that configures shading processes within the implemented pipeline, see Appendix A.2.

The shadeVertex implementation must call the **setPosition**⁸ function on the supplied vertex buffer with the transformed (unhomogenized) screen-space position as the parameter. This position is used by the clipper and rasterizer.

The computeColor implementation must return the color to be written into the linebuffer/framebuffer.

It is recommended when developing a custom Shader that naming of its classes/structures follows "*nameType*", where "*name*" is the name a Shader is referred by and "*Type*" are the class/structure type names used above.

3.4 Attribute interpolation

Interpolation across triangles happens on a per-pixel basis and should therefore be done very efficiently. Barycentric coordinates are used to this end, as they can be incrementally and quickly interpolated [3, p.183]. For a triangle with vertices $[x_1, y_1]$, $[x_2, y_2]$, $[x_3, y_3]$, barycentric coordinates can be computed

⁷Doing so allows Integrated Development Environments (IDEs) to give type hints when developing a custom Shader, and allows checking the uniform use of the same ShaderInterface at compile-time.

⁸The setPosition function is a member of BaseVertexBuffer.

by:

$$\begin{aligned}\alpha &= \frac{f_{23}(x, y)}{f_{23}(x_1, y_1)} & \beta &= \frac{f_{31}(x, y)}{f_{31}(x_2, y_2)} & \gamma &= \frac{f_{12}(x, y)}{f_{12}(x_3, y_3)} \\ f_{23}(x, y) &= (y_2 - y_3)x + (x_3 - x_2)y + x_2y_3 - x_3y_2 \\ f_{31}(x, y) &= (y_3 - y_1)x + (x_1 - x_3)y + x_3y_1 - x_1y_3 \\ f_{12}(x, y) &= (y_1 - y_2)x + (x_2 - x_1)y + x_1y_2 - x_2y_1\end{aligned}$$

Since these are all evaluations of $f(x, y) = Ax + By + C$, interpolating, e.g., α in x by d_x is simply:

$$\alpha = \frac{f_{23}(x + d_x, y)}{f_{23}(x_1, y_1)} = \frac{f_{23}(x, y)}{f_{23}(x_1, y_1)} + \frac{A}{f_{23}(x_1, y_1)} d_x = \frac{f_{23}(x, y)}{f_{23}(x_1, y_1)} + C_1 d_x \quad (3.1)$$

Barycentric interpolation of attributes a_1, a_2, a_3 requires $a = a_1\alpha + a_2\beta + a_3\gamma$. Interpolating in x by d_x can therefore be done using a single addition:

$$a = a + (a_1 \frac{y_2 - y_3}{f_{23}(x_1, y_1)} + a_2 \frac{y_3 - y_1}{f_{31}(x_2, y_2)} + a_3 \frac{y_1 - y_2}{f_{12}(x_3, y_3)}) d_x = a + C_2 d_x \quad (3.2)$$

Same applies for interpolation in y .

Incremental barycentric attribute interpolation is provided in *MicroRenderer* by the **TriangleAttribute** class. It is templated with:

- A numeric data type used for all internal computations, which must be float or double. Must match the Renderer's numeric data type.
- The attribute's type, which can be float, double or the library's VectorX types templated with float or double.

TriangleAttribute provides three functions:

- **initialize**: computes the initial attribute value and prepares interpolation. Its parameters are three attributes, barycentric coordinate increments⁹ and an initial offset from the first vertex supplied as parameters.
- **increment**: allows efficient interpolation by either one or by a given increment in x or in y , depending on the function's template enumeration **IncrementationMode**. IncrementationMode options: OneInX, OneInY, OffsetInX, OffsetInY
- **getValue**: returns the current attribute value.

⁹ C_1 of Equation (3.1) and its five analogous constants.

The `TriangleAttribute` class is meant to be used as member of a user's `TriangleBuffer`. The Shader's `setupTriangle` provides already computed barycentric coordinate increments and an offset to use as parameters for calling the attribute's `initialize` function. The Shader's `interpolateAttributes` function must then call their increment functions to perform interpolation. To this end `interpolateAttributes` is templated with an `IncrementationMode` parameter, which must be used with the increment functions. This way the `Renderer` class can execute all types of interpolations efficiently, without requiring the user to implement them separately.

3.5 Input/Output data representation

3.5.1 Models

MicroRenderer is limited to triangles as geometric primitives. A model built from triangles is described through five elements:

- An array of **VertexSource** structures, containing per-vertex information, see Section 3.3. The aggregation of all source data belonging to a vertex together in one structure, as opposed to supplying individual data channels in separate arrays, should help avoid cache-misses.
- A signed 32-bit integer containing the number of vertices (the length of the `VertexSource` array).
- An array of **TriangleIndices** structures defining triangles. They contain three unsigned 16-bit integers, each of them an index into the `VertexSource` array.
- A signed 32-bit integer containing the number of triangles (the length of the `TriangleIndices` array).
- A templatable **ModelData** object, initialized with the above four elements, acting as the model representation for the `Renderer` class. This structure is defined as a member of the `ShaderInterface` class and must be templated likewise to be usable for rendering.

Models are embedded into an application's source code as instances for these five elements in a C++ header file. They can be made `constexpr` to allow a compiler writing them into Read Only Memory (ROM), e.g., the MCU's flash memory.

The `Renderer` class must be given an array of accordingly specialized `ModelData` instances, representing the models that can be rendered.

3.5.2 Textures

In order to support different color-formats found in displays, *MicroRenderer* implements a **Texture2D** class abstracting access to a memory buffer for images. The target memory and the resolution of a

Texture2D instance can be set at runtime, and the class supplies a variety of read and write functions for coordinate-based and incremental access. Incremental access is utilized by the *Renderer* when writing into linebuffers/framebuffers for efficient access across one scanline. Values can also be sampled at non-integer coordinates using bilinear-filtering through the **samplePixelAt** function. Texture2D is templated with:

- A numeric data type used for all internal computations, which must be float or double. Must match the *Renderer*'s numeric data type.
- An instance of the **TextureConfiguration** structure that configures various aspects of the class, see Appendix A.3.

Pre-made textures can be embedded into an application's source code as five variables in a C++ header-file: a *TextureAccess* and a *TextureInternalFormat* variable, see Appendix A.3, two signed 32-bit integers describing texture width and height, and an array containing the image data, whose type depends on the texture's format. The data array of a read-only texture can be made `constexpr` to allow a compiler writing it into ROM.

3.5.3 Conversion tools

Two Python scripts are provided to assist the user in creating model and texture description headers. They can import commonly used file formats (e.g., obj, fbx, jpeg, png), convert and write their data according to a respective accompanying ini configuration file to produce valid C++ headers. The model converter can write vertex position, normal and multiple color- and UV-channels in user-defined order to fit a custom *VertexSource*¹⁰. The texture converter can adjust the imported image to fit any desired texture format available in *MicroRenderer*.

3.6 Pipeline in-depth

The pipeline implemented in *MicroRenderer* is described in detail in this section, following the basic structure of Figure 2.1. Geometry Processing and some part of Triangle Setup are performed for each object to be rendered in series; the remaining steps then happen for all objects at once.

¹⁰Color-channels define scalar attributes, while UV-channels define two-dimensional vector attributes.

3.6.1 Geometry Processing

Vertex Shading

For all vertices of an object the `shadeVertex` function is called, followed by homogenizing the clip-position if perspective projection is configured. The homogenized clip-position is stored separately in this case, since both are needed for correct clipping and rasterization, as mentioned in Section 2.3.1. To reduce divisions, the inverse of the depth is computed and then multiplied with the vertex positions for homogenization.

Transformations are part of the `shadeVertex` function and therefore responsibility of the user, but it shall be mentioned that is sensible to combine transformation-matrices as much as possible. Ideally only one matrix-vector multiplication should be made. Additional four multiplications can be saved for every initial vertex position transform, by skipping multiplications with the homogeneous component that always starts out as one. The **`transformPosition`** function of the `Matrix4` class should be used for this purpose.

Clipping

To avoid unnecessarily processing triangles that are never seen, two additional *Culling*¹¹ steps are performed before clipping:

First, all triangles facing away from the viewer are removed, a process called *Backface culling*. The front-facing side of a triangle is specified by its *vertex-winding-order*, i.e., if the triangle's vertices appear clockwise or counter-clockwise on screen, in the order they are supplied. It is computed for a triangle $[v1, v2, v3]$ with positions in screen-space by:

$$area = (v2.x - v1.x) \times (v3.y - v1.y) - (v2.y - v1.y) \times (v3.x - v1.x)$$

This area is negative for a front-facing clockwise-ordered triangle and positive for a counter-clockwise one, so culling can be performed by discarding a triangle based on its area's sign. To additionally discard almost degenerate triangles¹² not suited for rasterization, the area is actually compared against a threshold value, instead of checking its sign. The winding-order to cull can be configured in the `Renderer` class to match the used models.

The second culling step removes triangles that are either completely above, below, to the left, or to the right of the visible screen window, using a procedure based on the Cohen–Sutherland clipping-algorithm [11]:

¹¹Culling refers to techniques used to remove completely invisible triangles early to save computations.

¹²A degenerate triangle has an area of zero.

1. Compute 4-bit integer codes, called *outcodes*, for each of the three vertices of the triangle. Every bit of the outcode is set depending on the vertex's screen-space position relative to the screen window, according to:

	left of	inside	right of
above	1001	1000	1010
inside	0001	0000	0010
below	0101	0100	0110

2. Take the bitwise-AND of the three outcodes. If the result is non-zero, the triangle is fully outside the screen and discarded.

Clipping triangles analytically is an expensive process, so it is best avoided. Clipping at the *near-plane* of the view-volume¹³ is necessary, as mentioned in Section 2.3.1, and is done with the following procedure for every triangle using unhomogenized clip-positions:

1. Count number of triangle's vertices beyond the near-plane, i.e., potentially visible.
2. If the count equals 0: Skip triangle.
3. If the count equals 3: Render triangle as is.
4. If the count equals 1: Designate vertex visible v1. Designate vertices invisible v2 and v3, maintaining original winding order after v1. Interpolate new vertices c1, c2 to intersection of near-plane with v1-v2 and v1-v3 edge. Render new triangle made from v1-c1-c2.
5. If the count equals 2: Designate vertex invisible v1. Designate vertices visible v2 and v3, maintaining original winding order after v1. Interpolate new vertices c1, c2 to intersection of near-plane with v1-v2 and v1-v3 edge. Render new triangles made from v3-c2-c1 and v3-c1-v2.

Step 4. and 5. are visualized in Figure 3.1.

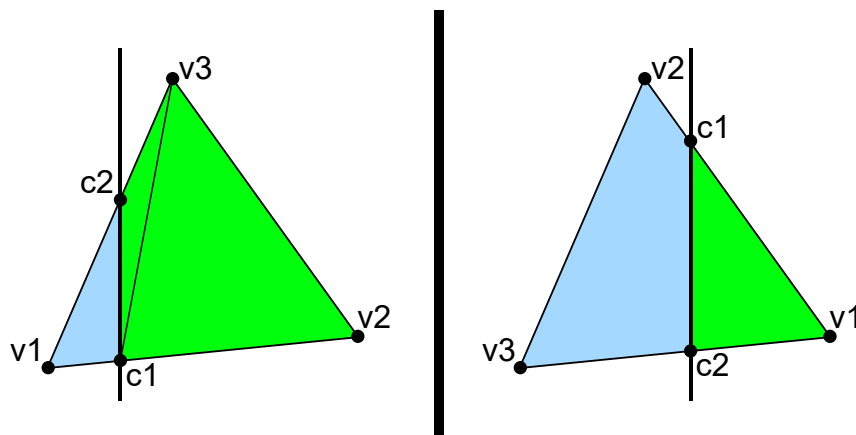


Figure 3.1: Visualization of non-trivial clip cases. Resulting triangles are colored in green.

¹³Plane designations used in this section are as shown in Figure 2.2.

Since the Rasterization stage contains bound-checks to prevent accessing pixels off-screen, clipping at the *left*, *right*, *top*, and *bottom-plane* can safely be skipped. Clipping at the *far-plane* is done per-pixel when solving hidden-surface removal, described in Section 3.6.3.

Screen Mapping

The screen mapping sub-stage of Section 2.3 can be expressed as a matrix and applied together with the previous transformations as well. To save additional calculations, the user is required to include screen mapping as part of their transformation(s) in the `shadeVertex` function. The pipeline assumes the coordinate system visualized in Figure 3.2 in screen-space, with the transformed view-volume at $[-0.5, s_x - 0.5] \times [-0.5, s_y - 0.5] \times [0, 1]$, where s_x and s_y are the screen's width and height.

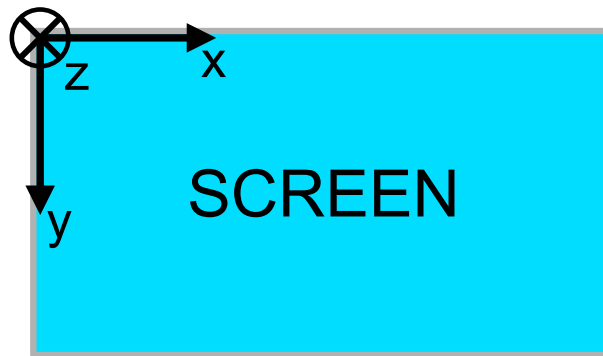


Figure 3.2: The coordinate system in screen-space assumed by *MicroRenderer*.

3.6.2 Rasterization

Most scanline-rasterization methods are made to support arbitrary polygons and hence operate on their line segments [4]. As a consequence, redundant information like references to owning polygons/objects or line slopes are stored multiple times. The method developed for and used in *MicroRenderer* targets triangles only, allowing it to reduce memory requirements and accesses. Its basic idea is outlined next, for complete details refer to the `Renderer` class in *MicroRenderer*'s source code [12].

Triangle Rasterization: Triangles are rasterized in two parts, called half-triangles, separated by the horizontal line going through the middle-vertex along y . The start scanlines of the half-triangles are computed and the positions of their edges on these scanlines, as well as their edge slopes. Starting with the first half-triangle, its covered scanlines are rasterized in series by coloring the pixels between the edge positions, and increasing the positions by the edge slopes after each line. The second half-triangle proceeds in the same way, only required to change one edge position and slope when starting its rasterization, since one edge is shared by the half-triangles. Each of the half-triangles needs a minimum

width in y to avoid numerical errors, so depending on the triangle's layout there may be just one half-triangle or none at all (in this case the triangle is discarded).

Triangle-pixel coverage: A pixel is covered by a triangle when its midpoint is inside the triangle's edges. If its midpoint is exactly on an edge, i.e., on the edges of two different triangles, it must be deterministically assigned to one of them. Otherwise the rendered image will depend on the rasterization order of triangles, which changes during movement of the view-point due to the sorting process that happens later. To this end, *MicroRenderer* applies the *Bottom-Right-Rule* (analogous to the *Top-Left-Rule* [1, pp.995-996]):

A pixel that lies on an edge is covered by the triangle if:

- The edge is horizontal and the other edges of the triangle are above it, or
- The edge is non-horizontal and on the right side of a triangle.

Instead of testing edges on this rule, *MicroRenderer* implicitly enforces it by shifting left and top start points of triangles by a tiny amount when computing half-triangle edge-positions and the first half-triangle's start scanline.

Triangle Setup

For every triangle that has passed clipping/culling, the members of a **RasterizationBuffer** structure, are computed:

- Data required for triangle rasterization.
- An index into the Renderer class's InstanceData buffer as a reference to the triangle's object.
- A horizontal position number used by interpolation.
- The triangle's TriangleBuffer instance, stored here to avoid additional indirection and cache-misses.

These structures are stored in a buffer of the Renderer class, which must be supplied by the user together with the buffer's size. The fixed winding order of vertices can be exploited to infer their relative position to another, depending on the half-triangle's layout, which saves comparison operations when computing rasterization data. Cases where half-triangles don't cover a single scanline or peak outside the screen are handled separately.

After filling the RasterizationBuffer, the `setupTriangle` function is called for the triangle, providing it the embedded TriangleBuffer and a start position for attribute interpolation. Then a **RasterizationOrder** structure, stored in another buffer of the Renderer class that must be supplied by the user, is filled with:

- The first scanline covered by the triangle as a signed 16-bit integer.
- An index into the RasterizationBuffer array, pointing to the corresponding entry there, as an unsigned 16-bit integer.

After all triangles of all objects have been processed, this array is sorted by its entries' first scanline, in rising order. Since the structures occupy only 32-bit, they can be efficiently copied during sorting with a single CPU instruction.

The amount of RasterizationBuffer and RasterizationOrder instances needed can't be determined exactly, as it depends on the amount of culled and clipped triangles. In the worst case it can be more than the total number of triangles in the scene if all are inside the view-space, oriented towards the viewer and some are also clipped, resulting in additional triangles. In practice the required number will often be less than the total triangle-count, but it is up to the user to supply the RasterizationBuffer and RasterizationOrder buffers with an appropriate size. Should rendering need more structures than available, any further triangles will be discarded.

Triangle Traversal

During traversal an *active section* in the RasterizationOrder array is tracked, expressed by a start and stop index, that denotes the triangles visible on a scanline. Scanlines are now rasterized and shaded from the screen's top to bottom into two Texture2D instances that serve as linebuffers for color and depth, which are members of the Renderer class, using the following procedure:

1. Add newly visible triangles to the active section by incrementing the stop index. A triangle is newly visible if its first scanline is less or equal than the current scanline.
2. For every triangle in the active section, using their RasterizationBuffers:
 - (a) Shade it on current scanline, see below.
 - (b) If its first half-triangle has ended on this scanline, update its RasterizationBuffer for the second half-triangle, if present. This operations only needs to swap data inside the RasterizationBuffer.
 - (c) If the whole triangle ended here, remove it from the active section by swapping its RasterizationOrder with the one at the start of the active section and incrementing the start index.
3. Update the linebuffers' memory addresses to a new region (must be done by the user). The depthbuffer can remain the same memory, but has to be cleared to zero (or a negative number). The user may want to initially clear the colorbuffer with any values.

A half-triangle is shaded on a scanline using the following procedure:

1. First pixel on scanline is half-triangle's left edge position rounded up, but at least the first pixel.
2. Last pixel on scanline is half-triangle's right edge position rounded down, but at most the last pixel.
3. If the last pixel is before the first pixel, end procedure here.
4. Interpolate attributes in x from the last pixel's x-coordinate of the last scanline, stored within the RasterizationBuffer, to this scanline's first pixel's x-coordinate¹⁴. This is done by calling the Shader's interpolateAttributes function with IncrementationMode OffsetInX.
5. Overwrite the last pixel's x-coordinate entry in the RasterizationBuffer with this scanline's last pixel's x-coordinate.
6. Get the first pixel's memory addresses in both linebuffers using its coordinates. They are the first target addresses of the first target pixel.
7. Go to Item 10.
8. Interpolate attributes by one in x by calling the Shader's interpolateAttributes function with IncrementationMode OneInX.
9. Increment the target addresses in the linebuffers by one pixel in x (the new target pixel).
10. If the target pixel passes the depth-test, compute its color and write the color and depth at the target addresses in their linebuffers, see Section 3.6.3.
11. If the pixel just drawn was not the last pixel, go to Item 8.
12. Increment the half-triangle's edge positions by their edge slopes.
13. Interpolate attributes by one in y by calling the Shader's interpolateAttributes function with IncrementationMode OneInY.

3.6.3 Fragment Processing

Fragment Shading

A pixel's color is computed and returned by the Shader's computeColor function. It is only called if the pixel passes the depth-test, see below.

Merging

Hidden-surface removal is handled with a *Reversed-Z* depthbuffer, which means that one minus the depth is stored and used for comparison. Because the normalized screen-space depth is inversely-proportional to view-space depth when using perspective-projection, its precision reduces the farther points are away from the image-plane. Reversed-Z offers a much more uniform depth distribution in

¹⁴Triangle setup initially interpolated attributes to the first scanline's last pixel and stored this position in the RasterizationBuffer, so this works on a triangle's first scanline as well.

comparison [1, pp.100-101].

The reversed depth of a pixel has been set up and interpolated automatically by pre-implemented functionality in the Shader's `setupTriangle` and `interpolateAttributes` functions. The type used for depth is the underlying numeric type of the `Renderer`.

The depth-test can be used for automatic per-pixel clipping at the far-plane, by initializing the `depthbuffer` with zeros. Initializing it with a very large negative value disables far-plane clipping.

3.7 Library Usage

A user can gain access to all of *MicroRenderer*'s classes and functionality by including the "MicroRenderer/MicroRenderer.h" collective header file.

All rendering is executed by calling functions on an instance of the `Renderer` class, specialized with the desired underlying data type, renderer configuration and Shader to use.

Initialization is performed by calling:

- **setResolution** with the desired output resolution.
- **setModels** with the `ModelData` array of the scene's models.
- **setInstances** with the `InstanceData` array of the scene's instances and its length.
- **setGlobalData** with a pointer to the `GlobalData` instance.
- **setVertexBuffer** with the `VertexBuffer` array (buffer).
- **setRasterizationBuffers** with the `RasterizationBuffer` and `RasterizationOrder` arrays (buffers) and their length.

Continuous rendering can then be done by repeatedly performing an `UPDATE` and a `DRAW` step.

In the `UPDATE` step the user may update any data necessary to reflect a changed scene before a new frame is rendered. Usually this will include changing values in the `InstanceData` array or changing its length to remove/add objects from/to rendering. Any settings made during initialization may be changed as well, if necessary.

The `DRAW` step does the actual rendering and depends on the `RenderMode` of the `Renderer`'s `RendererConfiguration`.

If the `RenderMode` is `FRAMEBUFFER`, a cleared frame- and `depthbuffer` with space for the full image must be first set by calling **setFramebuffer** and **setDepthbuffer** respectively. Then the **render** function must be called, which renders the entire image into the buffers.

If the `RenderMode` is `SCANLINE`, the **render** function is called first to setup rendering, then the **renderNextScanline** function needs to be called repeatedly, an amount of times equal to the output height

set with **setResolution**. Before every call to **renderNextScanline**, the user must supply cleared linebuffers for color and depth with enough space for one image row of the output width set by **setResolution**, using **setFramebuffer** and **setDepthbuffer** respectively.

Chapter 4

Evaluation

To give an idea of *MicroRenderer*'s performance, frame render times have been measured for two distinct Shader implementations on three different platforms at different resolutions. The test platforms, Shaders and measurements, complete with example images, are discussed in this chapter. The shader code used is also published alongside *MicroRenderer* [12].

4.1 Test platforms

The devices used for evaluation are:

1. **RP2040 MCU by the Raspberry Pi Foundation** [6]

This device is selected as a low-end, low-cost example MCU, featuring dual ARM Cortex-M0+ cores and 264kB on-chip RAM. Tests are run with the device clocked at 250MHz. Especially notable is the lack of a hardware FPU, although optimized software-routines for faster floating point calculations are embedded in ROM [6, Section 2.8.3.2.]. Note that while the RP2040 possesses dual-cores, all tests are run using one core only.

2. **STM32H747 MCU by STMicroelectronics** [8]

This device is selected as a high-end, high-cost example MCU, featuring an ARM Cortex-M7 core with a double-precision FPU with 16kB of both data and instruction cache and 1MB of on-chip RAM, as well as an external memory controller. Tests are run with the device clocked at 400MHz, with the frame- and depthbuffer stored in an external 32MB-SDRAM clocked at 100MHz and configured for write-through. Note that while the STM32H747 also possesses an ARM Cortex-M4 core, all tests are run on the Cortex-M7 core only.

3. **A Windows-based PC** While not meant as a target platform, the library can be used on non-MCU platforms as well, especially useful during Shader development. For comparison sake, the

tests are also run on a Windows-PC featuring an Intel Core i9-12900K CPU and DDR5 memory.

4.2 Test procedure

The same Shaders detailed below are run on all three target platforms, but rendering with different underlying data types and for different output resolutions and formats:

- The RP2040 renders using float at 120x120 pixels for 12-bit RGB color.
- The STM32H747 renders using float at 480x480 pixels for 24-bit RGB color.
- The Windows-PC renders using double at 1000x1000 pixels for 24-bit RGB color.

The ShaderConfiguration for every test case is configured to perform perspective projection, cull at screen borders, clip at the near plane, test for depth, and shade color. The Shader output type differ per test case, but the swizzle is always none.

The RendererConfiguration for every test case is configured to render in scanlines with front-facing triangles having clockwise winding-order (matching the used models).

A total of 3600 frames are rendered. Each rendered frame, all scene objects are rotated by 0.1° around their upwards center-axis, in order to vary the amount of covered screen space. The 'Measurements and Renders' sections below contain the minimum, maximum and average render times of all 3600 frames in milliseconds and in fps, as measured on each platform. The measured frame times are recorded on the MCUs using Timers configured for counting microseconds. The *<chrono>* C++ library is used for measurements on the Windows-PC. All test programs have been compiled with -O3 optimization.

4.3 Test Shader 1: SimpleContours

4.3.1 Shader explanation

This Shader defines a base color per object and attenuates it per-triangle based on a "sun-direction". The Shader's structures contain (additionally):

- GlobalData: Nothing.
- InstanceData: The model-to-screen-space matrix, a pointer to an array of pre-processed triangle normals, the normalized sun-direction in model-space, and the unnormalized base color.
- VertexSource: The model-space position.
- VertexBuffer: Nothing.
- TriangleBuffer: The triangle's final color.

The Shader's functions perform:

- `shadeVertex`: Transforms the source position.
- `interpolateVertices`: Interpolates the source positions.
- `setupTriangle`: Computes the triangle's final color according to:

$$color_{final} = color_{base} \times \max(0.2, (\mathbf{sun} \cdot \mathbf{normal} + 1) \times 0.5)$$

- `interpolateAttributes`: Nothing.
- `computeColor`: Returns the triangle's final color.

Shader output type is configured for `TYPE_DECIMAL`, i.e., the returned colors are unnormalized floating point values based on the bit-depth.

4.3.2 Scene composition

The test scene consists of two overlapping cube models and one model of the logo of TU Wien, built from a total of 856 vertices and 304 triangles.

4.3.3 Measurements and Renders

Table 4.1: Measured frame times of the SimpleContours Shader test scene.

	Frame Time in milliseconds			Frame Time in frames per second		
	Minimum	Average	Maximum	Minimum	Average	Maximum
RP2040	19.308	23.553	24.489	40.83	42.46	51.79
STM32H747	11.505	14.809	16.513	60.56	67.52	86.92
Windows-PC	0.487	0.632	0.958	1043.84	1582.61	2053.39

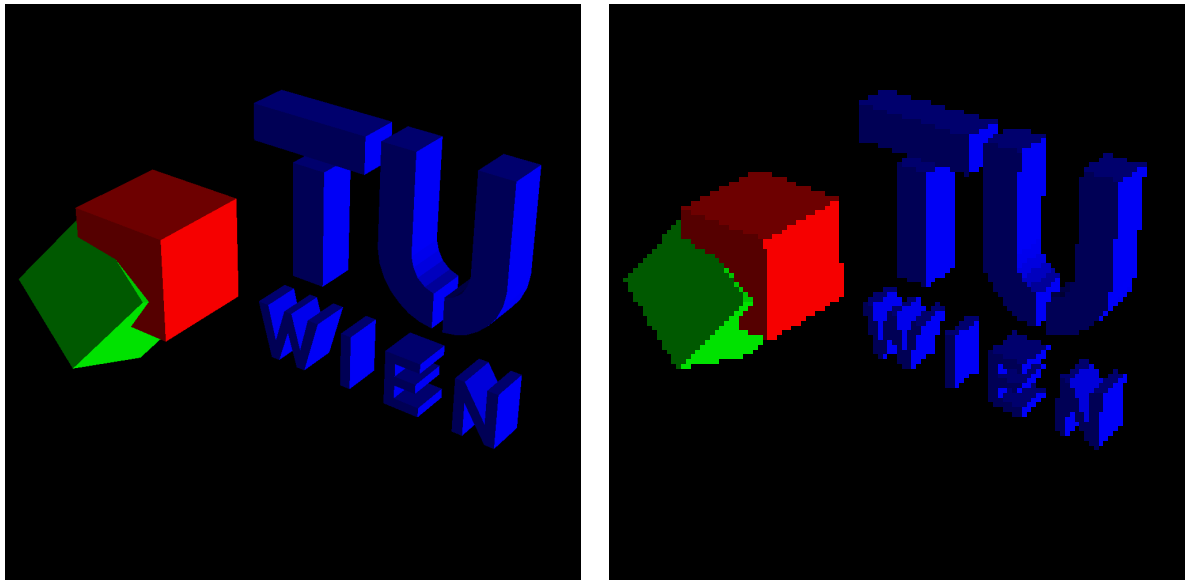


Figure 4.1: Renders of the SimpleContours Shader test scene, captured on PC. The left image was rendered at resolution 1000x1000. The right image was rendered at resolution 120x120.

4.4 Test Shader 2: UnlitTextured

4.4.1 Shader explanation

This Shader defines a texture per object and projects it onto the object's triangles, based on its vertices' UV coordinates¹. The Shader's structures contain (additionally):

- GlobalData: Nothing.
- InstanceData: The model-to-screen-space matrix and a Texture2D instance for the texture.
- VertexSource: The model-space position and the UV coordinate.
- VertexBuffer: Nothing.
- TriangleBuffer: A TriangleAttribute for the UV coordinate.

The Shader's functions perform:

- shadeVertex: Transforms the source position.
- interpolateVertices: Interpolates the UV coordinates.
- setupTriangle: Initializes the UV-TriangleAttribute.
- interpolateAttributes: Performs interpolation of the UV-TriangleAttribute.
- computeColor: Returns the nearest pixel in the texture at the UV-TriangleAttribute coordinate.

¹Two-dimensional texture coordinates are commonly called UV coordinates.

Measurements are given below for two different Shader output type configurations. One is using *TYPE_NORMALIZED*, i.e., the returned colors are normalized floating point values. The other uses *TYPE_INTEGER*, i.e., the returned colors are integer values based on the bit-depth.

4.4.2 Scene composition

The test scene consists of one cube model and two plane models built from a total of 44 vertices and 16 triangles. The planes cover the same space but face in opposite directions. The cube texture is a color grid of 3x3 resolution, projected on all six sides, while the planes' texture is the logo of TU Vienna at 128x49 resolution, see Figure 4.2. Both textures are configured for read-only access, no swizzle and no wrapping/clamping. The texture's format and type matches the shader output type of the respective test-case.



Figure 4.2: The textures used in the UnlitTextured Shader test scene, visualized in 24-bit RGB.

4.4.3 Measurements and Renders

Table 4.2: Measured frame times of the UnlitTextured Shader test scene with texture and shader output type *TYPE_NORMALIZED*.

	Frame Time in milliseconds			Frame Time in frames per second		
	Minimum	Average	Maximum	Minimum	Average	Maximum
RP2040	10.948	22.118	28.018	35.69	45.21	91.34
STM32H747	13.028	28.88	37.331	26.79	34.63	76.76
Windows-PC	2.545	5.815	9.123	109.61	171.96	392.93

Table 4.3: Measured frame times of the UnlitTextured Shader test scene with texture and shader output type *TYPE_INTEGER*.

	Frame Time in milliseconds			Frame Time in frames per second		
	Minimum	Average	Maximum	Minimum	Average	Maximum
RP2040	7.207	13.694	17.004	58.81	73.03	138.75
STM32H747	10.951	24.52	31.686	31.56	40.78	91.32
Windows-PC	2.045	4.769	8.316	120.25	209.7	489

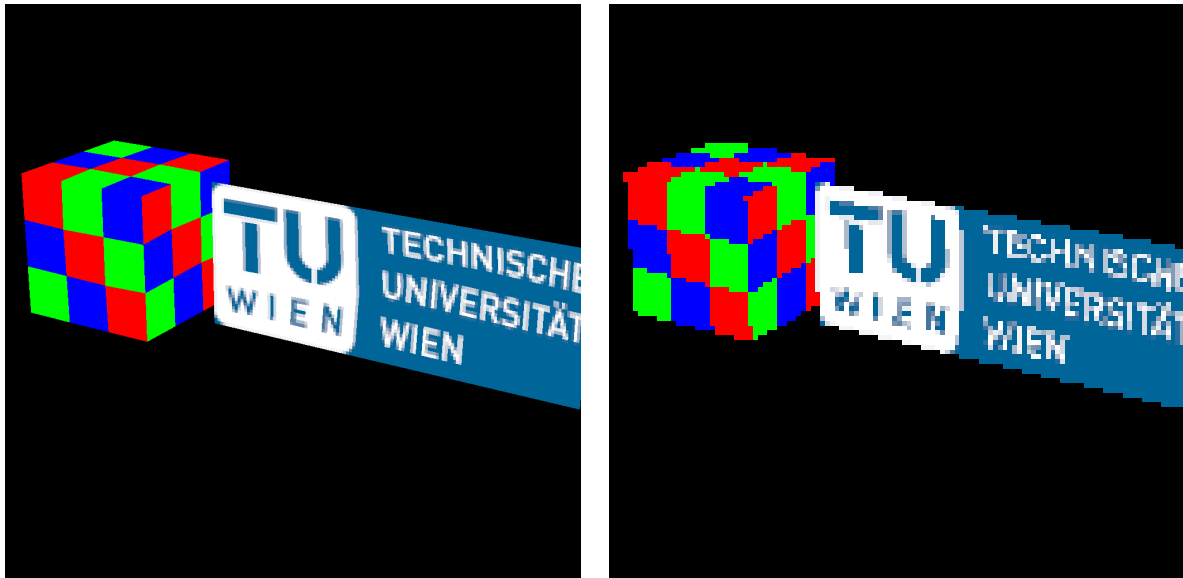


Figure 4.3: Renders of the UnlitTextured Shader test scene, captured on PC. The left image was rendered at resolution 1000x1000. The right image was rendered at resolution 120x120. Slight distortion in the applied textures are due to linear interpolation of UV coordinates in perspective screen-space.

4.5 Analysis

All test cases for both MCUs are running at above real-time frame rates, therefore enabling interactive 3D graphics of reasonable complex scenes. The large margins above 20-24 fps, especially on the more memory-constrained RP2040, allow the user to omit a full framebuffer to save RAM. Notable about the tests on the RP2040 are the longer render times with the SimpleContours Shader compared with the UnlitTextured Shader, which is contrary to the other platforms. The reason for this is the lower render resolution used with the RP2040, since per-pixel computations, only present in the UnlitTextured Shader, rise dramatically with screen dimension. The vertexShade operations coming with the larger amount of geometry in the SimpleContours test scene are therefore computationally more expensive than simple per-pixel operations on low-resolution screens.

The two distinct tests using different texture and Shader output types run with the UnlitTextured Shader show a key strength of the versatile configuration structure provided by textures. As no lighting computations are used, i.e., the colors read from the textures are put on screen unmodified, one may directly read from the texture and write into the colorbuffer using integers in the appropriate bit-depth, as done by using `TYPE_INTEGER`. Compared to `TYPE_NORMALIZED`, this saves calculations for pixel normalization, scaling and integer <-> floating point conversions². This simple change in Shader code brings an increase in fps by 17.8% on the STM32H747, 21.9% on the Windows-PC and a massive 61.5%

²`TYPE_NORMALIZED` is still useful due to its normalized output of course.

on the RP2040. The stronger effect on lower-end platforms is likely due to increased expenses for floating-point operations and the simpler processor architecture, preventing the compiler from hiding the additional operations.

Chapter 5

Conclusion and Outlook

MicroRenderer provides a quick and easy way to render 3D graphics on resource-constrained systems like MCUs. Like shader scripts on hardware GPUs, the design of the Shader Framework provides flexible access to a robust clipper and rasterizer, complete with easy accessibility to attribute interpolation. A user may even take Vertex and Fragment Shader scripts for a GPU and translate it quite directly to a set of Shader classes. *MicroRenderer*'s performance characteristics prove to be suitable for real-time rendering of simple scenes, even enabling the use of linebuffers for massive memory savings, therefore expanding its potential use cases to especially resource-constrained devices. The flexibility of the Shader Framework further allows usage for high-detail, non-real-time image generation.

There are numerous possibilities for continued work on *MicroRenderer*, both for further increasing render speed and adding additional features, for example:

- *Deferred Shading*, where only triangle references are stored in the colorbuffer during rasterization, and the fragment shade computations are performed afterwards, avoiding computing pixel-colors that are later overdrawn.
- Pre-processing models into *Spatial data structures*, which can be used to discard triangles even earlier in the pipeline, e.g., BSP-Trees.
- Hidden-surface removal by analytically determining triangle intersections per scanline, instead of using a depth-buffer. This saves further memory and may also compute faster for simpler scenes, since per-pixel depth interpolation and read/write is omitted. Intersection tests for more complex scenes may, however, end up decreasing performance.
- Simple *Anti-Aliasing* of triangle edges, giving object edges a smoother appearance. This may be incorporated relatively easily together with the rasterization procedure used.
- Perspective-correct attribute interpolation, which would normally require a division per pixel. Only performing correct interpolation in intervals and linearly interpolating between them may

give a better look at less drastic computational requirements.

- Adding native support for lines and points as a geometric primitive.
- Adding native support for *Quads* as a geometric primitive, possibly improving performance for heavily quad-based models. Quads also exhibit less distortions with affine attribute interpolation when using perspective projection.

Bibliography

- [1] T. Möller, E. Haines, N. Hoffman, A. Pesce, M. Ilwanicki, and S. Hillaire, *Real-time rendering*, fourth edition. ed. Boca Raton: CRC Press, 2018.
- [2] Arm, *Cortex-M0 Technical Reference Manual*, Arm, 2009.
- [3] S. Marschner and P. Shirley, *Fundamentals of computer graphics*, fifth edition. ed. Boca Raton: CRC Press/Taylor and Francis Group, 2022.
- [4] M. M. Rahman, “Visibility computations – scanline algorithms and techniques,” in *Computational Science and Its Applications - ICCSA 2007, Part II: International Conference, Kuala Lumpur, Malaysia, August 26-29, 2007. Proceedings*, ser. Lecture Notes in Computer Science 4706, ICCSA 2007 Kuala Lumpur. Springer-Verlag Berlin Heidelberg, 2007, pp. 25–37.
- [5] Arm, *Arm Cortex-M4 Processor Technical Reference Manual*, Arm, 2020.
- [6] Raspberry Pi Foundation, *RP2040 Datasheet*, Raspberry Pi Foundation, 2024.
- [7] Interplay, *Descent (User manual)*, Interplay, 1995.
- [8] STMicroelectronics, *STM32H747xI/G Datasheet (DS12930)*, STMicroelectronics, 2023.
- [9] Arm, “Arm cortex-m processor comparison table,” Arm, 2023.
- [10] T. Möller and B. Trumbore, “Fast, minimum storage ray/triangle intersection,” in *ACM SIGGRAPH 2005 Courses*. New York, NY, USA: ACM, 2005, pp. 7–7–es.
- [11] J. D. Foley, *Computer graphics : principles and practice*, 2nd ed., ser. The systems programming series. Boston, Mass. [u.a.]: Addison-Wesley, 1999.
- [12] D. Pusztai. (2024) Microrenderer source code. [github.com]. Last Accessed: September 24, 2024. [Online]. Available: <https://github.com/ronypepper/MicroRenderer>

Appendix A

Configuration Structures

All configuration structures utilized in *MicroRenderer* and their effects are listed here in detail.

A.1 **RendererConfiguration**

The **Renderer** class receives this structure to configure the rendering pipeline.

Its members are:

- A **RenderMode** enumeration:
Determines if rendering should be done per scanline.
Options: FRAMEBUFFER, SCANLINE
- A **FrontFace** enumeration:
Determines which side of a triangle is culled, see Section 3.6.1.
Options: COUNTERCLOCKWISE, CLOCKWISE
- A **ShaderConfiguration** structure: See Appendix A.2

A.2 **ShaderConfiguration**

The **Shader** classes and the **Renderer** class (via **RendererConfiguration**) receive this structure to configure shading processes within the rendering pipeline.

Its members are:

- A **ProjectionType** enumeration:
PERSPECTIVE enables homogenization of transformed vertex positions and adds an additional homogeneous clipping position to **BaseVertexBuffer**. This is necessary since clipping requires unhomogenized positions, while rasterization needs homogenized ones. PERSPECTIVE requires

ClippingMode to be CLIP_AT_NEAR_PLANE.

Options: ORTHOGRAPHIC, PERSPECTIVE

- A **CullingMode** enumeration:

Enables/disables culling triangles fully outside screen.

Options: CULL_AT_SCREEN_BORDER, NO_CULLING

- A **ClippingMode** enumeration:

Enables/disables clipping triangles at near-plane.

Options: CLIP_AT_NEAR_PLANE, NO_CLIPPING

- A **DepthTestMode** enumeration:

Enables/disables the depth-test. DEPTH_TEST_DISABLED removes the depth TriangleAttribute from BaseTriangleBuffer (no depthbuffer can be set in this mode) and requires ShadingMode to be SHADING_ENABLED.

Options: DEPTH_TEST_ENABLED, DEPTH_TEST_DISABLED

- A **ShadingMode** enumeration:

Enables/disables execution of the computeColor function. SHADING_DISABLED may be used to render only depth values of the scene and requires DepthTestMode to be DEPTH_TEST_ENABLED.

Options: SHADING_ENABLED, SHADING_DISABLED

- A **ShaderOutput** structure containing:

- A **TextureInternalFormat** enumeration:

Sets the underlying format of the framebuffer, see Appendix A.3.

- A **TextureSwizzle** enumeration:

Sets the underlying swizzle of the framebuffer, see Appendix A.3.

- A **TextureExternalType** enumeration:

Sets the access type of the framebuffer, see Appendix A.3.

A.3 TextureConfiguration

The Texture2D class receives this structure to configure various aspects within the class.

Its members are:

- A **TextureAccess** enumeration:

Disables/enables writing to the texture.

Options: ACCESS_READONLY, ACCESS_READWRITE

- A **TextureInternalFormat** enumeration:

Determines the layout of pixels in memory. The numbers indicate the bit-depth of the corresponding color channel. `FORMAT_DEPTH` uses the numeric datatype the `Texture2D` class is templated with as its underlying type, i.e., it provides image-based access to a numeric array. Used by the depthbuffer with the `Renderer`'s underlying numeric type.

Options: `FORMAT_RGB888`, `FORMAT_RGB565`, `FORMAT_RGB444`, `FORMAT_RGBA4444`, `FORMAT_R8`, `FORMAT_R16`, `FORMAT_R32`, `FORMAT_DEPTH`

- A **TextureSwizzle** enumeration:

Swaps color channels. Only supported for RGB formats at the moment.

Options: `SWIZZLE_NONE`, `SWIZZLE_BGR`

- A **TextureExternalType** enumeration:

Determines the numeric return type of read-functions. With `T` as the `Texture2D` class's underlying numeric type, `TYPE_NORMALIZED` returns normalized `T`, `TYPE_DECIMAL` returns unnormalized `T` and `TYPE_INTEGER` returns unnormalized integers. The latter two can be used to save computations during shading, by avoiding normalizing, re-scaling and rounding operations. For example, when using `FORMAT_RGB888` the color of an object could be defined as three unsigned integers and directly written to the texture, without any floating-point operations in between.

Options: `TYPE_INTEGER`, `TYPE_DECIMAL`, `TYPE_NORMALIZED`

- A **TextureWrapmode** enumeration:

These define the behaviour of the read-functions when accessing out-of-bounds pixel coordinates. `WRAPMODE_NONE` does not check the for out-of-bound access at all,

`WRAPMODE_CLAMPING` clamps access to the texture's set resolution and

`WRAPMODE_TILING` wraps the coordinates around the texture in tiles.

Options: `WRAPMODE_NONE`, `WRAPMODE_CLAMPING`, `WRAPMODE_TILING`

Erklärung

Hiermit erkläre ich, dass die vorliegende Arbeit ohne unzulässige Hilfe Dritter und ohne Benutzung anderer als der angegebenen Hilfsmittel angefertigt wurde. Die aus anderen Quellen oder indirekt übernommenen Daten und Konzepte sind unter Angabe der Quelle gekennzeichnet.

Die Arbeit wurde bisher weder im In- noch im Ausland in gleicher oder in ähnlicher Form in anderen Prüfungsverfahren vorgelegt.

Copyright Statement

I, Dominik Pusztai, hereby declare that this thesis is my own original work and, to the best of my knowledge and belief, it does not:

- Breach copyright or other intellectual property rights of a third party.
- Contain material previously published or written by a third party, except where this is appropriately cited through full and accurate referencing.
- Contain material which to a substantial extent has been accepted for the qualification of any other degree or diploma of a university or other institution of higher learning.
- Contain substantial portions of third party copyright material, including but not limited to charts, diagrams, graphs, photographs or maps, or in instances where it does, I have obtained permission to use such material and allow it to be made accessible worldwide via the Internet.

Signature: Dominik Pusztai

Vienna, Austria, October 2024

Dominik Pusztai